# cryptoone (TVXCTFCRYPTO1) - Cryptography Writeup

## Prompt

> ender chest
>
> You found a world seed nobody could guess and used it as the key to his
> chests. Elliptic curves are unbreakable, so the curve the server admin
> picked must be fine, right?... RIGHT???

Supplied in cryptoone.7z: TVXCTFCRYPTO1/chall.py and
TVXCTFCRYPTO1/output.txt. params.py and the flag files are not supplied.

## Flag

```
TVX{D1dn5_F4lL_4_7h3_7r4p_7h15_t1me}
```

Decoy in the same file - do not submit this one:

```
TVX{tNt_g035_B00m_C0ngr4t5}
```

## What chall.py does

```
from params import p, a, b, G, d
FLAG     = open("flag.txt").read().strip()
REALFLAG = open("realflag.txt").read().strip()

def chest(msg, key_src):
    key = hashlib.sha256(key_src).digest()
    iv  = os.urandom(16)
    return (iv + AES.new(key, AES.MODE_CBC, iv).encrypt(pad(msg.encode(), 16))).hex()

Q = d * G
ender_chest = chest(FLAG,     str(d).encode())    # opens with the world seed
shulker_box = chest(REALFLAG, str(Q[0]).encode()) # opens with the x coordinate
```

So there are two AES-256-CBC ciphertexts in output.txt
(iv ? ciphertext, hex), with keys SHA256(str(d)) and SHA256(str(Q_x))
on the curve $y^2 = x^3 + a \cdot x + b \pmod p$:

```
p = 472486135597375986246648585213775 9          (112-bit prime)
a = 92756691649605926479232674650726 4
b = 47159683304708407707457816307035 60
G = (36759422040920472832381362530930 45, 35447270854946576034307184041728 45)
Q = (38694181758384503190545047855259 59, 14290602533388882347706682166898 96)
```

Both G and Q check out on the curve, and the discriminant
$4a^3 + 27b^2 ? 0 \pmod p$, so the curve is not singular.

## Step by step

### 1. The shulker box is a trap

Its key source is Q[0] - the x-coordinate of the public key, printed in
plain text in output.txt. Anyone can open it, no cryptanalysis at all:

```
key = SHA256(b"38694181758384503190545047855259 59")
```

```
shulker_box -> TVX{tNt_g035_B00m_C0ngr4t5}
```

"TNT goes boom" is the joke: this is the exploding chest. The variable in chall.py is even called REALFLAG to talk you into stopping here.
The real flag is in the ender chest, and that one needs the secret d.

## 2. The curve the admin picked is not fine - its order is smooth

p is only 112 bits, but generic ECDLP (Pollard rho, ~2?? steps) would still be out of reach, and d is not small (no match in the first 2,000,000 multiples of G). The actual flaw is the group order.

Counting points (Schoof's algorithm - PARI/GP ellcard) gives:

```
#E = 4724861355973759917094976238449899
   = 11 x 367 x 13399 x 58453 x 484079 x 40128631 x 76927309
```

ellgroup reports the group as cyclic with that order, and G has order exactly #E - G is a generator. Every prime factor is below 2²?.
That is a textbook Pohlig-Hellman curve: the ECDLP is only as hard as its largest prime factor.

The solver does not have to trust the point count. With N = 4724861355973759917094976238449899 it checks in pure Python that

- N·G = O, and
- (N/q)·G ? O for every prime q | N,

which together prove ord(G) = N exactly. Factoring N itself is also easy (Pollard rho), precisely because all of its factors are small.

## 3. Pohlig-Hellman: recover d

For each prime factor q? of N, project the problem down to the subgroup of order q?:

```
N? = N / q?
G? = N?·G          (order q?)
Q? = N?·Q = (d mod q?)·G?
```

Each small discrete log is solved by baby-step / giant-step (?q? steps - at most ?76,927,309 ? 8,772 for the largest factor):

q? | d mod q?

11 | 3

367 | 341

13,399 | 6,052

58,453 | 1,439

484,079 | 88,736

40,128,631 | 22,612,983

76,927,309 | 44,817,997

CRT over the seven residues gives the single value modulo N:

```
d = 232756972273944770824637756980339 5
```

Verified directly: d·G == Q, exactly, on the given curve.

### 4. Open the ender chest

```
key = SHA256(str(d)) = SHA256(b"232756972273944770824637756980339 5")
iv  = first 16 bytes of ender_chest
AES-256-CBC decrypt, strip PKCS#7
    -> TVX{D1dn5_F4lL_4_7h3_7r4p_7h15_t1me}
```

"Didn't fall for the trap this time" - the flag text itself confirms
which of the two chests was the real one.

## Solver

```
solvers/solve_01_cryptoone.py   # this challenge, standard library only
solvers/solve_all.py            # runs every solver in the package
```

The solver is standard-library only - it carries a small pure-Python
AES-256 decryption and does its own EC arithmetic, BSGS, Pollard-rho
factoring and CRT - and it prints the decoy as well as the flag so the
trap stays visible:

```
$ python3 solvers/solve_all.py
=== solve_01_cryptoone.py ===
[*] shulker_box (key = Q[0], public) -> TVX{tNt_g035_B00m_C0ngr4t5}   <-- decoy, not the flag
[*] #E = ord(G) = 472486135597375991709 4976238449899
[*] factorisation = {11: 1, 367: 1, 58453: 1, 13399: 1, 484079: 1, 76927309: 1, 40128631: 1}  (all
 prime factors < 2^27: smooth)
[*] verified: N*G = O and (N/q)*G != O for every q | N, so ord(G) = N
[*] recovered d = 232756972273944770824637756980339 5
[*] check: d*G == Q -> True
Flag: TVX{D1dn5_F4lL_4_7h3_7r4p_7h15_t1me}
[+] solve_01_cryptoone.py: flag verified

1/1 challenges solved
```

Fresh-unzip test: solve_all.py passes 1/1 in under a second.

## Lesson

Two separate failures, either of which sinks the scheme on its own:

1. Never key anything from a public value. SHA256(Q_x) looks like a
   key derivation step, but Q_x is published, so the "encryption" is
   just an encoding - and here it was used to plant a convincing decoy.
2. A curve is not safe because it uses elliptic curves. Security rests
   on the largest prime factor of the base point's order. When the order
   is smooth, as here (largest factor ~2²?), Pohlig-Hellman turns the
   discrete log into a handful of trivial ones, whatever the size of p.
   Use a standard curve with a large prime-order subgroup, and check the
   order before trusting a custom one.